Процессы в IT-команде

Agile и Scrum · встречи спринта · роли · бэклог, груминг и оценка · путь задачи · code review · рабочий день

01 Agile → Scrum → спринт

Agile — подход: короткие итерации, частый показ результата, готовность к изменениям (ценности манифеста, а не правила). **Scrum** — самый популярный Agile-фреймворк. **Спринт** — фиксированные 1–4 недели (обычно 2) под цель спринта: длину не меняют, недоделанное возвращается в бэклог.

Бэклог продукта (ведёт PO) → **Планирование** (что берём + цель спринта) → **Бэклог спринта** (задачи + цель) → **Спринт** (2 недели, ↻ дейли) → **Инкремент** (готово по DoD) → **Демо** (продукт, с заказчиками) → **Ретро** (процесс, только команда)

↺ следующий спринт: фидбэк с демо и решения ретро идут в работу

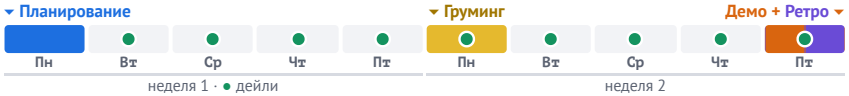
3 роли
Product Owner · Scrum Master · Developers

5 событий
спринт · планирование · дейли · демо · ретро

3 артефакта
бэклог продукта · бэклог спринта · инкремент

Kanban — без спринтов: непрерывный поток и **WIP-лимиты** (не больше N задач в колонке — сначала доделай). Метрика — время задачи (cycle time). Для поддержки и багфикса.

02 Встречи двухнедельного спринта



Планирование · 2 ч	что берём в спринт, цель спринта
Дейли · 15 мин	сделал · сделаю · что мешает. Синхронизация, не отчёт
Груминг	уточнить, раздробить, оценить задачи **следующего** спринта
Демо	показать заказчикам сделанное по DoD, собрать фидбэк
Ретро	только команда: хорошо / мешало / изменим → action items

Демо — про продукт, **ретро** — про процесс. Пример ретро: «PR висят по 3 дня» → дежурный ревьюер, ревью ⩽ 1 дня, ответственный — Олег → через спринт проверили.

03 Роли: кто за что

Product Owner
«что и в каком порядке»: бэклог, приоритеты, бизнес

Scrum Master
«как работаем»: процесс, встречи, помехи. Не начальник

Developers
все, кто делает: бэк, фронт, QA, аналитик. Сами оценивают

Иерархии нет. В жизни ещё: **тимлид** — техника, ревью · **аналитик** — требования · **QA** — качество · **DevOps** — CI/CD · **PM** — сроки

ДЖУНУ: К КОМУ ИДТИ

Смысл задачи, правила	→ аналитик / PO
Как сделать технически	→ тимлид / ментор
Как воспроизвести баг	→ QA
Сроки, блокеры	→ на дейли, SM / PM

Правило 15 минут: застрял — 15–30 мин пробуй сам, потом спроси, рассказав, что уже попробовал.

04 Бэклог и задача



Бэклог — упорядоченный список работы по продукту: фичи, баги, техдолг.

эпик → история → задача · баг

User story: Как **покупатель**, я хочу **повторить прошлый заказ**, чтобы **не собирать корзину заново**.
Критерий приёмки: *дано* части товаров нет, *когда* жму «Повторить», *тогда* в корзину попадает остальное и видна плашка.

DoR — готова **к взятию**: история, критерии, макет, оценка

DoD — **готова**: ревью, зелёные тесты, проверка QA

05 Груминг и оценка задач

Груминг (backlog refinement) — посреди спринта: уточняем требования и критерии, дробим крупное, оцениваем. К планированию задачи готовы (DoR).

1	2	3	5	8	13	21	?	☕

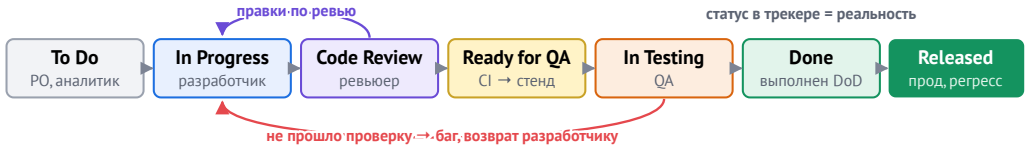
Story points — относительная сложность: объём + сложность + **неопределённость**. Эталон «поле в API» = 2 SP, остальное — относительно него.

PLANNING POKER

1. PO или аналитик рассказывает задачу, команда спрашивает
2. каждый тайно выбирает карту, открывают **одновременно**
3. большой разброс → объясняют авторы **мин.** и **макс.** оценки
4. переголосовывают, пока не сойдутся

> 13 → дробить. Velocity — SP за спринт, на неё набирают план. SP сравнивают только внутри одной команды.

06 Путь задачи: кто с кем взаимодействует



ОТВЕТ «ПО ПУТИ ЗАДАЧИ»

1. **PO, аналитик:** история и критерии приёмки
2. **Груминг:** dev и QA спрашивают, оценивают
3. **Dev:** код → PR → ревью → мерж → стенд
4. **QA:** проверка по критериям → баг → фикс → **ретест**
5. **Менеджерам** — риски и блокеры **заранее**: дейли, чат

БАГ-РЕПОРТ

- шаги воспроизведения
- ожидаемый ↔ фактический результат
- окружение: стенд, версия, браузер
- скриншот, логи
- **severity** (насколько серьёзно) ≠ **priority** (как срочно)

07 Code review

Проверка кода коллегами **до мержа в main** — в pull request (merge request).

1. **Ветка** от main → **PR:** что, зачем, как проверить
2. **CI:** тесты, линтер
3. **Ревью:** комментарии ↺ правки
4. **Approve** → merge

блокер — нельзя мержить · **nit** — мелочь · **вопрос** — хочу понять

Зачем: ошибки и уязвимости до прода · единый понятный код · знания у всей команды · общая ответственность

Маленький PR, про код — не про человека, ревью в течение дня.

08 Рабочий день

- 9:30 чат, CI, план на день
- 10:00 **дейли** 15 мин
- 10:15 **фокус:** код, тесты, отладка
- 13:00 обед
- 14:00 **ревью** коллег, ответы на комментарии
- 14:45 встречи, синк с QA и аналитиком
- 15:30 **фокус**
- 18:00 статусы в Jira, план на завтра

Код ≈ 4–5 ч, встречи 1–2 ч, ревью 30–60 мин, остальное — переписка

Статус в чат: **вчера / сегодня / блокеры**. Сразу суть, без «есть минутка?».

LLM: как устроены и как с ними работать

01 Токены и контекстное окно

`При` `вет` `,` `как` `де` `ла` `?` 7 токенов

`Hello` `,` `how` `are` `you` `?` 6 токенов · условный токенизатор

Токен — кусок текста по словарю токенизатора: слово, часть слова, знак. В английском ≈ 4 символа на токен, русскому на ту же мысль обычно нужно больше. В токенах считают **цену, лимиты и окно**. Токен LLM ≠ JWT-токен.

◂ вход: промпт — **контекстное окно** — общий бюджет на один вызов — выход ▸

system	история диалога	найденные документы (RAG)	вопрос	**резерв под ответ**	свободно

Не влезло → ошибка API, обрезка старой истории или **оборванный ответ** (`finish_reason = "length"`). Длинный контекст дорогой, а детали в середине модель замечает хуже.

02 Как модель пишет текст — и почему галлюцинирует

Модель считает **вероятности следующего токена**, выбирает один, дописывает и повторяет:

«Столица Франции — »

Париж	0.91
город	0.04
это	0.02
Лион	0.01

↻ выбранный токен дописывается к тексту → снова предсказание, пока не «стоп» или лимит

Почему галлюцинирует

- ищет **правдоподобное**, факты не проверяет — базы фактов нет
- не знает ваших и свежих данных (дата среза)
- «не знаю» — не самый вероятный ответ
- случайность выбора, мутный контекст

Как снизить

- RAG: ответ по источнику со ссылками
- разрешить ответ «не знаю»
- tool calling для живых данных
- низкая temperature, проверка ответа

03 temperature, top-p, top-k



T = 0.2 острый пик — **T = 1** как обучили — **T = 1.5** почти ровно

temperature меняет «остроту» распределения. Низкая — стабильно: факты, код, JSON. Высокая — разнообразно, но больше ошибок. **T = 0 ≠ без ошибок.**



top-k = 3 только k лучших — **top-p = 0.9** набрать сумму до p

Отсекают маловероятные токены. Крутят обычно что-то одно.

04 Роли в LLM API

`SYSTEM` Ты — поддержка магазина. Отвечай кратко.

`USER` Где мой заказ 123?

`ASSISTANT` → вызов `get_order(123)`

`TOOL` {"status": "в пути"}

`ASSISTANT` Заказ в пути.

system — правила, роль и формат от разработчика, приоритет выше. **user** — запрос в этих рамках.

API **не помнит** историю: шлём её в каждом запросе → токенов всё больше. System — не защита от prompt injection: права проверяет код.

05 Few-shot и chain of thought

Zero-shot — без примеров:
`Тональность: «Доставили за час!»`

Few-shot — 2–5 примеров «вход → выход» задают формат и границы:
«Курьер опоздал» → негатив
«Всё пришло целым» → позитив
«Доставили за час!» → ?

Chain of thought — «рассуждай по шагам, потом ответ». Точнее на многошаговых задачах, но дороже и медленнее.

Reasoning-модели рассуждают сами; рассуждение отделяют от ответа.

06 Стриминг ответа



без стриминга — со стримингом

↑ ~0.4 с (TTFT) — весь ответ ~8 с

```
data: {"delta": "Зак"}
data: {"delta": "аз в пути"}
data: [DONE]      # упрощённо
```

Время то же, но **первые слова сразу**. Куски идут по **SSE** (`stream=True`). Для чата и длинных ответов; JSON разбирают, когда собран целиком. Обрыв посередине надо обработать.

07 Гарантированно валидный JSON

1. Просьба в промпте — гарантии нет

2. JSON mode — синтаксис валиден, поля — нет

3. Structured outputs — JSON-схема ограничивает саму генерацию

Ответ temperature 0 → **finish_reason** "stop", не "length" → **Pydantic** форма + бизнес-правила → **✓ в код** разбор, роутинг

```
class Ticket(BaseModel):
    category: Literal["оплата", "доставка", "другое"]
    priority: Literal["low", "high"]
    summary: str = Field(max_length=200)
```

✗ ошибка → повтор с текстом ошибки валидации, **не больше N раз** → фолбэк (категория «другое», ручной разбор). Повтор — страховка, гарантию даёт схема.

08 Как снизить стоимость

(вход × цена + выход × цена) × запросы

выходные токены обычно в разы дороже

- **модель по задаче**, каскад: дешёвая → сильная
- **меньше входа:** RAG вместо всей базы, сжатие истории, короткий system
- **меньше выхода:** `max_tokens`, краткий формат
- **кэш** ответов (и семантический), кэш промпта у провайдера
- **Batch API** для несрочного
- **учёт и лимиты** токенов, следить за ретраями и циклами агентов

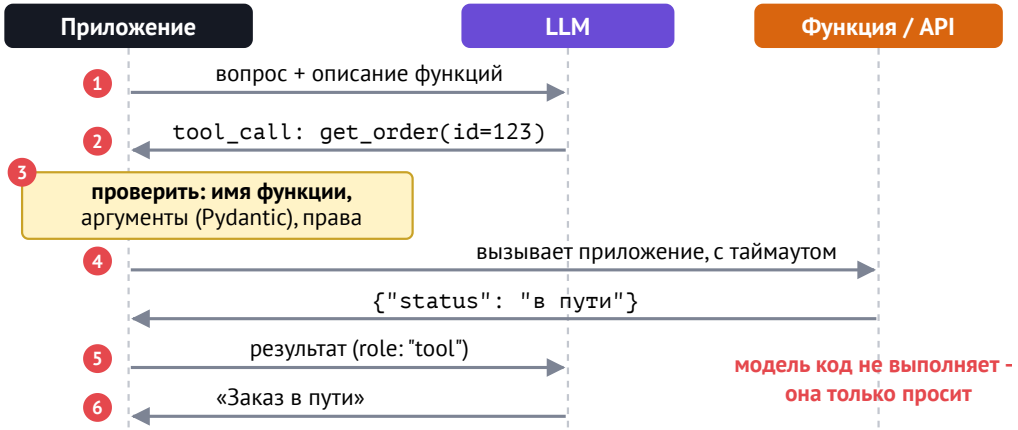
09 Открытые и закрытые модели

	ЗАКРЫТЫЕ (API) GPT, CLAUDE, GEMINI	ОТКРЫТЫЕ ВЕСА LLAMA, QWEN, DEEPSEEK
Качество	обычно вершина рейтингов	догоняют; для узких задач хватает
Цена	за токены, старт без вложений	GPU; выгодно при большом объёме
Данные	уходят провайдеру	не покидают твой контур
Контроль	модель могут обновить или снять	версия своя, дообучай как хочешь
Запуск	ничего поднимать не надо	Ollama, vLLM, облачный хостинг

Прототип и сложные задачи → API. Чувствительные данные → открытая в своём контуре. Модель — за своим адаптером, чтобы менять провайдера.

Tool calling и агенты

01 Tool calling



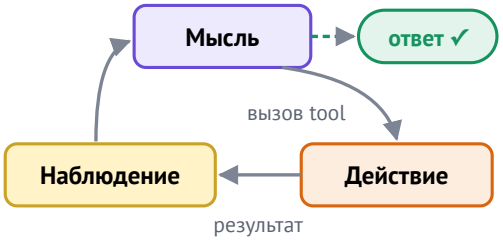
Модель вместо текста возвращает **имя функции и аргументы в JSON** по описанию инструментов. Вызов, валидация, права и таймауты — на приложении. Результат возвращают модели, и она пишет ответ. Один ответ может содержать несколько tool_call.

02 Агенты: ReAct и Plan-and-Execute

Обычный вызов LLM
один запрос → один ответ текстом; шаги задаёт код

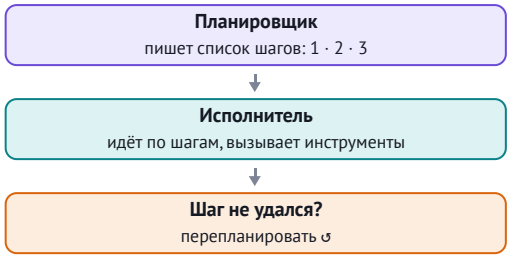
Агент — LLM **в цикле с инструментами**: сама выбирает следующий шаг, действует, смотрит на результат и продолжает, пока не достигнет цели

`REACT: ДУМАЙ → ДЕЙСТВУЙ → СМОТРИ`



`Мысль: нужен статус → get_order(123) → «в пути» → ответ`

`PLAN-AND-EXECUTE: СНАЧАЛА ПЛАН`



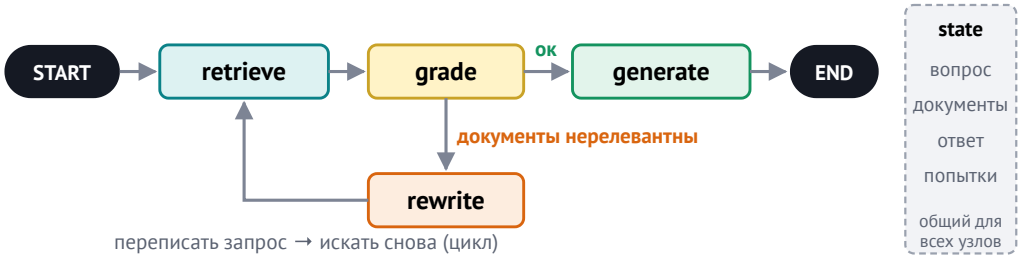
Для длинных задач: план видно и можно проверить заранее.

Ограничители обязательны: лимит шагов и токенов, таймауты, подтверждение опасных действий человеком (оплата, удаление), логи каждого шага.

03 LangChain и LangGraph

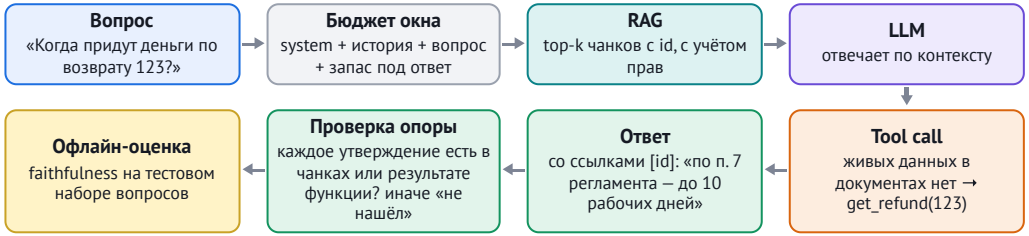
LangChain — готовые блоки: модели, промпты, загрузчики, сплиттеры, ретриверы, цепочки. Быстро собрать RAG и сменить провайдера.

LangGraph — агент как **граф**: узлы, общий **state**, условные переходы, **циклы**, сохранение состояния, человек в контуре.



Простой линейный сценарий — **обычный код**. Ветвления, циклы, долгие агенты с паузой на человека — **LangGraph**.

04 Ассистент по документам + функция: как всё связано



1. **Токены:** окно и цена не дают положить всю базу знаний в промпт
2. **RAG** выбирает несколько релевантных кусков — только из того, что пользователю можно видеть
3. **Tool calling** — за живыми данными (статус, остатки); модель просит, приложение вызывает
4. **Опора на источник:** ссылки [id] на переданные чанки + офлайн-метрика **faithfulness** на тестовом наборе

05 Частые ошибки

- ~~«Модель сама вызывает функцию»~~ → только просит, вызывает приложение
- ~~«Агент — любой чат-бот»~~ → цикл с инструментами и решениями модели
- ~~«Чем больше чанков, тем лучше»~~ → шум и деньги
- ~~«RAG — это дообучение»~~ → подкладывает найденное в промпт
- ~~«Проверили на 5 вопросах»~~ → тестовый набор и метрики

LangChain и tool calling

01 Что проходит по цепочке

Запрос **«Можно вернуть заказ №42?»** проходит через три шага. Смотри на тип данных после каждого.

Шаг	Вход → выход
Шаблон	`ChatPromptTemplate` подставляет вопрос и, если предусмотрено, историю. На выходе `ChatPromptValue` с сообщениями и ролями.
Модель	`ChatModel` читает сообщения и возвращает `AIMessage` с текстом или запросами инструментов.
Парсер	`StrOutputParser` извлекает текст из финального `AIMessage`.

```
# Обычный ответ текстом
chain = prompt | model | StrOutputParser()
text = chain.invoke({"question": question})
```

LCEL соединяет шаги оператором `|`. Выход слева становится входом справа. Вся цепочка тоже **Runnable**, то есть её можно запускать через тот же интерфейс, что и отдельный шаг.

Разобрать типы на примере урока.

02 Invoke, batch и stream

Вызов	Когда нужен
`invoke()`	Один вопрос → один результат. Удобно посмотреть, что вернул каждый шаг.
`batch()`	Список вопросов → список результатов. Например, проверить 10 заявок за один запуск кода.
`stream()`	Один вопрос → части одного ответа по мере готовности. Например, печатать текст в чате.

`batch` обрабатывает несколько независимых входов. Это общий интерфейс LangChain. Количество сетевых запросов зависит от реализации модели.

Чтобы видеть поток в `stream`, поддержка потоковой выдачи нужна и на последующих шагах. Шаг, который сначала собирает весь вход, задержит выдачу.

03 Схема ответа и инструмент

Задача	Механизм
Получить поля `order_id`, `eligible`, `reason`	**Structured output** задаёт схему финального ответа.
Узнать возраст заказа №42	**Tool calling** даёт модели описание `get_order`. Модель запрашивает вызов, приложение выполняет его.

В LangChain схему можно передать через `with_structured_output`. **Валидация** проверяет, что поля есть и подходят по типам. `eligible=true` может пройти схему, даже если по регламенту возврат запрещён. Факты проверяем по заказу и найденному правилу.

У аргументов инструмента своя схема. Перед вызовом код проверяет имя, параметры и доступ к заказу.

Повторить схемы и контракты.

LangChain и tool calling

04 Кто вызывает функцию

```
AIMessage → get_order(order_id=42)
            id вызова = call_1
Код → выполняет get_order(42)
ToolMessage → 40 дней, не использован
              tool_call_id = call_1
Модель → читает результат, решает дальше
```

`model.bind_tools(tools)` передаёт модели описания функций. Код читает `AIMessage.tool_calls`, находит разрешённую функцию и выполняет её. Результат добавляется как **ToolMessage** с тем же `tool_call_id`.

Заказ даёт факты о покупке. Если нужно узнать правило, модель запрашивает `search_docs`. Когда запросов инструментов больше нет, приложение может извлечь финальный текст парсером.

Посмотреть полный цикл.

05 Что модель получает снова

Так растёт список сообщений в учебном запуске. S - инструкция, U - вопрос, A - запрос инструмента от модели, T - его результат.

Запрос к модели	Что в списке
1. Выбрать первый шаг	`S, U` - 2 сообщения
2. После проверки заказа	`S, U, A1, T1` - 4 сообщения
3. После поиска правил	`S, U, A1, T1, A2, T2` - 6 сообщений

Да, код повторно передаёт накопленные сообщения. В третьем вызове модель видит и возраст заказа из `T1`, и найденное правило из `T2`. Три чанка внутри `T2` могут занимать много токенов, хотя это одно сообщение.

В длинном диалоге приложение выбирает, какую историю включать. Пара запроса и результата инструмента должна оставаться связанной.

06 Как проверить поиск агента

`search_docs` ищет чанки. **Векторный поиск** сравнивает эмбеддинги. **Гибридный** объединяет векторную выдачу и BM25, который помогает с точным кодом вроде `ERR-4031`.

RRF объединяет списки по позициям. За место `rank` чанк получает `1 / (60 + rank)` в этом примере. Вклады из списков складываются. Первое место даёт `1/61`, второе - `1/62`.

Golden set - вопросы с заранее отмеченными релевантными чанками. Для одного вопроса нужны A и B. Поиск вернул A, C, D.

Метрика	Расчёт
Precision@3	Из трёх найденных подходит один → `1/3`
Recall@3	Из двух нужных нашли один → `1/2`

Повторить RRF · Повторить метрики поиска.

Агенты и LangGraph

01 Кто выбирает следующий шаг

Возьмём один вопрос **«Можно вернуть заказ №42?»**.

Подход	Кто задаёт маршрут
Chain	Код заранее задаёт шаги. Проверить заказ → найти правило → ответить.
Workflow	Код задаёт маршруты и условия. Если заказ не найден, спросить номер. Если найден, проверить правило.
Agent	Модель выбирает следующий инструмент по задаче и предыдущим результатам. Код ограничивает доступные действия.

ReAct - выбрать действие, получить результат, снова выбрать действие или ответить. **Plan-and-Execute** - сначала составить план из нескольких шагов, потом выполнять и при необходимости уточнять его.

Сравнить подходы на одной задаче.

02 Что находится в State

State - рабочие данные текущего диалога или задачи. **Node** - функция, которая читает эти данные и возвращает обновление. Например, `check_order` добавляет результат проверки.

```
# До узла
{"order_id": 42, "attempts": 0}
# Узел вернул только изменённые поля
{"attempts": 1, "age_days": 40}
# После объединения
{"order_id": 42, "attempts": 1,
 "age_days": 40}
```

Reducer задаёт способ объединения значений одного поля. По умолчанию новое значение заменяет старое. Для сообщений `add_messages` добавляет новые ID и заменяет сообщение при совпадении ID. Поля, которые узел не вернул, сохраняются.

Посмотреть State и reducer в коде.

03 Куда идёт граф

Edge соединяет шаги. **Conditional edge** выбирает маршрут по состоянию. Например, роутер читает последнее сообщение модели.

```
START → model
model → END    (лимит исчерпан)
model → END    (нет вызова инструмента)
model → tools  (есть вызов и остались попытки)
tools → model  (результат записан в State)
```

В ручном цикле Python это были `while`, проверка `tool_calls` и вызов функции. В **StateGraph** это узлы и рёбра. `compile()` собирает запускаемый граф, `invoke()` запускает его с входными данными.

Лимит попыток в State даёт контролируемый выход. `recursion_limit` служит запасным ограничителем. Если граф превышает допустимое число шагов, возникает `GraphRecursionError`.

Агенты и LangGraph

04 Где сохраняется прогресс

Термин	Что означает
Checkpoint	Снимок состояния и прогресса выполнения на границе шага графа.
Checkpointer	Механизм записи и чтения снимков. Память удобна для демо, постоянное хранилище переживает перезапуск процесса.
`thread_id`	Ключ диалога или задачи, по которому загружаются её снимки.

Если открыть тот же `thread_id`, граф может продолжить с сохранёнными данными. Новый ID начинает отдельную историю, если для него ещё нет снимка.

State и запрос к модели - два разных набора данных. Граф может хранить заказ, счётчики и историю. В модель попадут только сообщения и другие данные, которые код включил в запрос.

Посмотреть, как сохраняется State.

05 Как поставить граф на паузу

Например, помощник подготовил возврат заказа №42. Перед операцией просим человека проверить её.

```
Подготовить → interrupt(черновик)
                ↓ пауза и снимок
Человек → одобрить / исправить / отклонить
                ↓ тот же thread_id
Command(resume=решение) → выбранная ветка
```

Для паузы нужен checkpointer. При возобновлении **узел запускается с начала**. Дойдя до `interrupt()`, он получает значение из `resume` и продолжает по коду.

После правки проверяем новые параметры. После отказа заканчиваем без операции. После одобрения переходим к отдельному узлу исполнения.

06 Как не провести возврат дважды

Если до `interrupt()` стоит запись во внешнюю систему, повторный запуск узла может выполнить её снова. Checkpoint фиксирует ход графа, а внешний сервис живёт по своим правилам.

Идемпотентность означает, что повтор той же операции сохраняет тот же результат без дубля. Например, `refund:order-42:req-17` - один и тот же ключ при каждой попытке провести конкретный возврат. Сервис возвратов проверяет ключ атомарно и возвращает прежний результат при повторе.

Проверка руками. Запусти запрос, останови граф на подтверждении, затем проверь ветки одобрения и отказа. Смоделируй сбой сразу после записи и повтори тот же запрос. У возврата должна остаться одна запись.

Разобрать повтор запуска.

Метрики ML: смысл, формулы и расчёт

01 Матрица ошибок

Классификация выбирает категорию; **регрессия** предсказывает число с размером ошибки. Macro/micro/weighted F1 - про классы; MAE/MSE/RMSE/R^2 - про числовые прогнозы. Score срочности ещё не делает классификатор регрессией.

Строки - **Fact (факт)**, столбцы - **Pred (предсказание модели)**. Positive (класс 1) - срочная заявка:

Fact / Pred	Срочная: Pred = 1	Обычная: Pred = 0
Срочная	TP = 8	FN = 2
Обычная	FP = 4	TN = 86

TP - нашли срочную; FP - обычную ошибочно назвали срочной; FN - срочную пропустили; TN - обычную верно оставили обычной.

Проверка: TP + FP + FN + TN = 100 заявок. Настоящих срочных: TP + FN = 10. Обычных: FP + TN = 90. Поэтому **TN = 90 - FP = 86**, а не количество пропущенных срочных.

Fact не меняется при выборе порога. Меняется Pred: кого модель назовёт срочным. Проверяйте подписи осей: в другой таблице их могут поменять местами.

02 Accuracy, Precision, Recall и F1

N - всего заявок; P - precision; R - recall. Все четыре метрики: **больше - лучше, 1 = 100%** (но полезность зависит от цены ошибок).

Метрика	Формула	Пример	Вопрос
Accuracy	(TP + TN) / N	94%	Как часто правы вообще?
Precision	TP / (TP + FP)	8/12 ≈ 66,7%	Доверять тревогам?
Recall	TP / (TP + FN)	8/10 = 80%	Сколько срочных нашли?
F1	2PR / (P + R)	72,7%	Каков их баланс?

Precision: среди названных срочными сколько настоящих? Берём **столбец Pred = 1**, включая FP. **Recall:** среди всех настоящих срочных сколько нашли? Берём **строку Fact = 1**, включая FN. Precision = 100% означает FP = 0; Recall = 100% означает FN = 0, если знаменатели ненулевые.

F1 = 2TP / (2TP + FP + FN) - гармоническое среднее: слабую составляющую трудно скрыть сильной. F1 не учитывает TN и цену ошибок. При редком классе «всегда обычная» даёт высокий Accuracy и нулевой Recall. Нулевой знаменатель требует явного соглашения.

03 Порог и цена ошибки

Правило: **positive, если score ⩾ t**. Score не обязательно калиброванная вероятность. На фиксированных scores снижение порога расширяет положительные предсказания: TP и FP не убывают, FN уменьшается или остаётся прежним. Recall не убывает; Precision может меняться в обе стороны.

Порог и ранжирование - разные вещи. При прежних scores смена t меняет TP/FP/FN/TN и Precision/Recall, но **не ROC-AUC и AP**: выбираем другую точку на прежних кривых. Срочная со score 0,8 выше обычной с 0,2; при t=0,9 обе будут обычными, при t=0,5 срочную найдём. Порядок верный в обоих случаях.

Цена = FP × цена ложной тревоги + FN × цена пропуска. При ценах 1 и 20 наш ущерб: 4 + 40 = 44. Порог выбирают по цене или ограничению, например Recall ⩾ 90%, на **validation**; **test** оставляют для финальной оценки.

Как посчитать нагрузку. Из 1 000 заявок 20 срочных, значит обычных 1 000 - 20 = 980. Пусть TPR = 0,8, FPR = 0,05. TPR умножаем на срочные, FPR - на обычные.

TP = 20 × 0,8 = 16; FN = 20 - 16 = 4. FP = 980 × 0,05 = 49; TN = 980 - 49 = 931. Очередь = TP + FP = 65 уведомлений. Precision = 16/65 ≈ 24,6%. Требование «recall ⩾ 0,8 и не более 50 уведомлений» не выполнено: полнота подходит, бюджет превышен. Всегда проверяйте оба условия.

AP: простой пример перед PR-кривой (§05). Оценивает, насколько рано находим настоящие positive в списке по убыванию score. Без одинаковых scores: складываем Precision при каждой находке positive и делим на число всех настоящих positive.

Простой пример: срочная → обычная → обычная → срочная. Находки на местах 1 и 4: Precision = 1/1 и 2/4. **AP = (1 + 0,5)/2 = 0,75.** Обычные учитываются в знаменателе следующей находки. Каждая находка даёт прирост Recall = 1/2, поэтому общая формула даёт то же: (1/2)×1 + (1/2)×0,5 = 0,75.

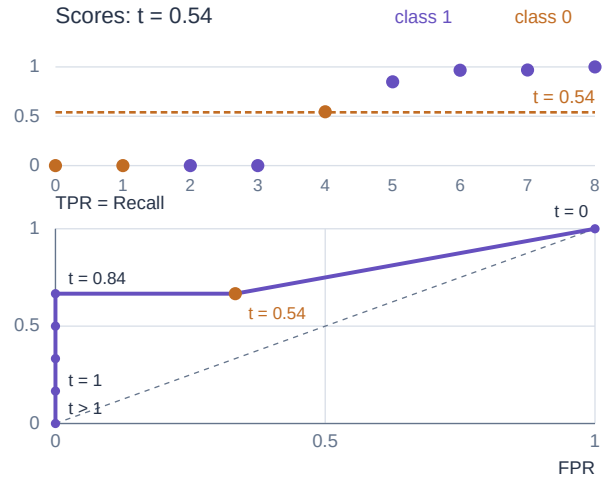
ROC-AUC сравнивает пары; AP учитывает Precision при находках. При редком positive высокая ROC-AUC не гарантирует мало ложных тревог: смотрите PR/AP.

Метрики ML: смысл, формулы и расчёт

04 ROC и ROC-AUC

Меняем порог t, а не scores. Если score ⩾ t, предсказываем 1. Одна точка - один порог; левее и выше лучше.

Пример из видео: 9 объектов - 6 срочных, 3 обычных. Scores и классы показаны на графике; правило score ⩾ t.



X = FPR = FP/3; Y = TPR = Recall = TP/6. Делим FP на 3 настоящих negative, TP - на 6 настоящих positive.

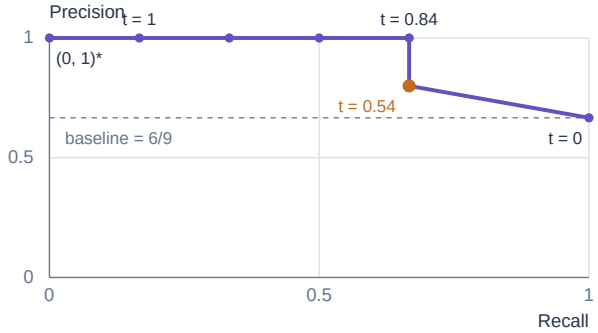
t	TP	FP	FPR	TPR
> 1	0	0	0	0
1	1	0	0	1/6
0,84	4	0	0	4/6
0,54	4	1	1/3	4/6
0	6	3	1	1

t=0,54: TP=4, FP=1, FN=2, TN=2; точка (1/3; 4/6). Нулевые scores включаем вместе при t=0. На кривой - все уникальные scores, в таблице - выбранные пороги.

ROC-AUC - площадь под ROC; тоже можно считать трапециями. Здесь (12 побед + 4 равенства × 0,5)/18 пар = **7/9 ≈ 0,778**. Сравниваем score срочной и обычной: 1 - идеальный порядок, 0,5 - случайный в ожидании. Не Accuracy; нужны оба класса.

05 Precision-Recall и Average Precision

Те же данные, те же пороги; другие оси. X = Recall = TP/(TP+FN); Y = Precision = TP/(TP+FP). Recall - какую долю класса 1 нашли; Precision - какая доля наших тревог верна.



* (Recall=0; Precision=1)

t	Recall	Precision
> 1	0	Не определена*
1	1/6	1/1
0,84	4/6	4/4
0,54	4/6	4/5
0	6/6	6/9

t=0,54: точка (Recall; Precision) = (2/3; 0,8). При t=0: Recall=1, Precision=2/3 - нашли всех, но треть тревог ложная.

***Без тревог Precision не определена:** знаменатель 0. Точка (Recall=0; Precision=1) добавлена только для начала графика, а не как оценка качества.

Общая формула: AP = Σ (Recall$_i$ - Recall$_{i-1}$) × Precision$_i$. Σ - сложить шаги; умножаем прирост Recall на Precision после шага. Равные scores включаем группой. Для девяти объектов выше: **AP = (4/6)×1 + (2/6)×(6/9) = 8/9 ≈ 0,889**.

PR-AUC по трапециям: ширина по Recall × средняя высота Precision; вертикали дают 0. Здесь **41/45 ≈ 0,911**, а AP ≈ 0,889: разные способы расчёта. Указывайте метод; это не процент верных ответов.

Пунктир - доля positive: 6/9. Сравнивайте PR/AP на одних данных: доля классов влияет на оценку.

06 Micro, macro и weighted

Где применяют. В бинарной классификации два класса (спам / не спам); в многоклассовой - три и больше (кошка / собака / птица). Усреднение применимо в обоих случаях: каждый класс рассматривают как «этот класс против остальных». В многометочной задаче у объекта несколько меток - усреднять тоже можно.

Когда усреднение не нужно. В бинарной задаче часто важен один positive, например спам: считают его Precision, Recall и F1 отдельно, без усреднения с «не спамом».

Пусть $F1_i$ - F1 класса i, n_i - число его настоящих объектов (**support**), K - число классов, N - сумма n_i. **Σ означает «сложить по всем классам».**

Способ	Формула для F1	Когда выбирать
Macro	Σ $F1_i$ / K	Каждый класс одинаково важен, включая редкий
Weighted	Σ (n_i × $F1_i$) / N	Нужно учесть реальную частоту классов; редкий может скрыться
Micro	2ΣTP / (2ΣTP + ΣFP + ΣFN)	Нужен общий итог по решениям; сначала складываем счётчики

Пример. Из 6 кошек все найдены; из 4 собак две названы кошками. Для кошек TP=6, FP=2, FN=0; F1=12/14≈0,857. Для собак TP=2, FP=0, FN=2; F1=4/6≈0,667.

Macro = (0,857 + 0,667)/2 ≈ **76,2%**. Weighted = (6 × 0,857 + 4 × 0,667)/10 ≈ **78,1%**. Micro: сумма TP=8, FP=2, FN=2; 16/(16+2+2) = **80%**.

Запомнить: macro - классы поровну; weighted - по численности; micro - общие счётчики. Macro-F1 усредняет готовые F1, а не средние P и R. Только при **одной метке на объект и учёте всех классов** micro-P = micro-R = micro-F1 = Accuracy. Для нескольких меток или части классов равенство не гарантируется.

Метрики ML: смысл, формулы и расчёт

07 MAE, MSE и RMSE

Регрессия предсказывает число, например время в минутах. **Размер промаха** - разница между прогнозом и фактом без минуса. Например, прогноз 18, факт 20: промах 2 минуты. Прогноз 22 при факте 20 - тоже промах 2 минуты.

MAE = сумма размеров промахов / число заявок. Показывает, на сколько минут в среднем ошиблись по модулю.

MSE = сумма квадратов промахов / число заявок. Сначала квадрат каждой ошибки, затем среднее. Большие промахи влияют сильнее. Единицы - минуты².

RMSE = √MSE. Берём квадратный корень из MSE, чтобы результат снова был в минутах. Большие промахи по-прежнему влияют сильнее.

Для всех трёх **меньше - лучше; 0 - идеально**. Разницу со знаком просто усреднять нельзя: +2 и -2 дадут 0, хотя в обоих случаях прогноз неточен.

Пример по шагам: факт [10, 20, 30], прогноз [12, 18, 40]. Всего 3 заявки.

Факт	Прогноз	Размер промаха	Квадрат промаха
10	12	2	$2^2 = 4$
20	18	2	$2^2 = 4$
30	40	10	$10^2 = 100$

MAE = (2 + 2 + 10) / 3 = 14/3 ≈ 4,67 мин.

MSE = (2^2 + 2^2 + 10^2) / 3 = (4 + 4 + 100) / 3 = 36 мин².

RMSE = √36 = 6 мин.

Если один промах вырос с 2 до 20, его размер вырос в 10 раз, а квадрат - с 4 до 400, в 100 раз. Это изменение одного слагаемого, а не множитель всей метрики: остальные промахи тоже участвуют в расчёте. При одинаковых данных и весах RMSE ⩾ MAE.

Эти метрики одинаково оценивают промах на 2 минуты вверх и вниз. Если недооценка дороже переоценки, отдельно проверяйте направление ошибок.

08 R^2 и базовый прогноз

R^2 сравнивает модель с прогнозом «всем среднее» по квадратам промахов. 1 - идеально; **0 - не лучше прогноза средним**, ниже 0 - хуже. Это не доля верных прогнозов и не минуты.

R^2 = 1 - (сумма квадратов промахов модели / сумма квадратов промахов прогноза средним). Для краткости первую сумму называют SSE, вторую - SST.

Среднее фактов = сумма фактических значений / число заявок. Базовый прогноз выдаёт всем это среднее.

Шаг 1. Среднее фактов: (10 + 20 + 30) / 3 = 20. Базовый прогноз: [20, 20, 20].

Шаг 2. С чем сравниваем? Ошибки простого способа: 20 - 10 = 10; 20 - 20 = 0; 20 - 30 = -10. Их квадраты: 100, 0, 100. **SST = 100 + 0 + 100 = 200.**

Шаг 3. Ошибки модели. Для прогноза [12, 18, 40] выше получены квадраты 4, 4, 100. **SSE = 4 + 4 + 100 = 108.** В формуле сравниваются суммы, без деления на число заявок.

Шаг 4. Не забыть «1 -»: 108/200 = 0,54 - оставшаяся доля ошибки. **R^2 = 1 - 0,54 = 0,46** - сумма квадратов ошибок уменьшилась на 46% относительно среднего. Не «46% прогнозов верны».

R^2 = 0,8 не гарантирует приемлемую ошибку. Если MAE = 30 минут при допустимых средних 5, модель не подходит. R^2 - сравнение со средним; MAE/RMSE - размер ошибки. Для требования «ни один промах не больше 5» одного MAE недостаточно.

R^2 = -0,3: сумма квадратов ошибок на 30% больше, чем у прогноза средним. Отрицательной ошибки в минутах здесь нет. R^2 разных наборов напрямую не сравнивайте: меняется разброс фактов.

При постоянной цели SST = 0, обычная формула не определена; библиотечная подстановка - отдельное соглашение. Среднее оцениваемых фактов нужно для расчёта R^2. Реальный прогноз для новых заявок строят по train, без знания будущих ответов.

09 Как выбрать метрику

Сначала определите тип ответа модели.

Задача / требование	Что смотреть
Категория; доверять тревогам	Precision
Категория; меньше пропусков	Recall
Баланс Precision и Recall	F1; цену ошибок проверить отдельно
Каждый класс одинаково важен	Macro F1
F1 с учётом частоты классов	Weighted F1
Общие TP, FP и FN классов	Micro F1
Качество списка по score	ROC-AUC и PR/AP; не выбранный порог
Число; средний размер ошибки	MAE в рублях, минутах и т. п.
Число; большие ошибки особенно нежелательны	RMSE в исходных единицах; MSE в квадратных
Число; лучше ли прогноза средним	R^2 вместе с MAE/RMSE

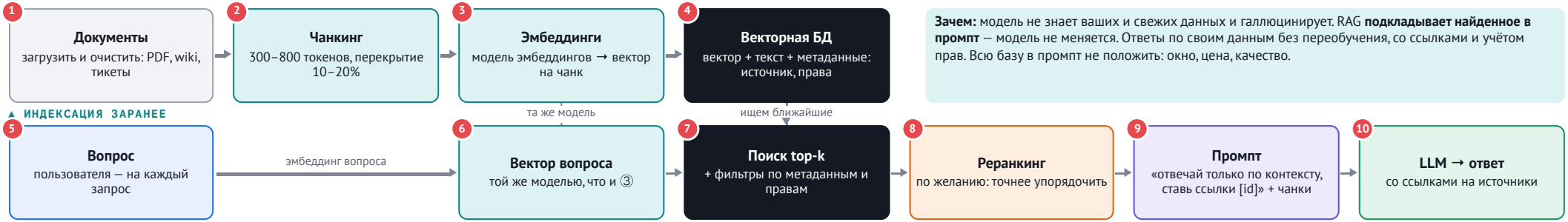
Сначала ограничения, потом лучший результат. Если команда проверяет максимум 50 тревог, вариант с 100 не подходит даже при лучшем Recall. Выберите максимальный Recall среди допустимых порогов.

Ответ ментору: тип задачи → positive и ошибки → знаменатель метрики → ограничение → числовой пример. В наборе из 99 обычных и 1 срочной заявки высокий Accuracy ещё не означает пользу. Ошибка 2 → 20 увеличивает вклад в MSE в 100 раз, в MAE - в 10 раз. Модель и порог подбирайте на validation, затем проверяйте на отложенном test без утечки.

RAG: поиск по своим документам

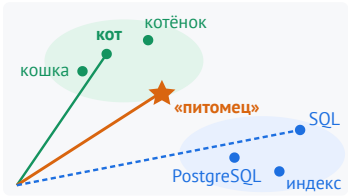
01 RAG-пайплайн по шагам

Retrieval-Augmented Generation: сначала найти нужные куски документов, потом ответить по ним



Зачем: модель не знает ваших и свежих данных и галлюцинирует. RAG **подкладывает найденное в промпт** — модель не меняется. Ответы по своим данным без переобучения, со ссылками и учётом прав. Всю базу в промпт не положить: окно, цена, качество.

02 Эмбеддинги



Эмбеддинг — вектор фиксированной длины (сотни–тысячи чисел). Его строит **отдельная модель эмбеддингов**.

Похожий смысл → близкие векторы, **даже без общих слов**: «питомец» рядом с «кот».

$$\cos(a, b) = a \cdot b \,/\, (|a| \cdot |b|)$$

малый угол → похоже (≈1), большой → не связано

Документы и вопросы — **одной** моделью. Где нужны: семантический поиск, RAG, кластеризация, рекомендации. Эмбеддинг ≠ токен.

03 Векторные базы данных

Векторная БД хранит векторы с текстом и метаданными и быстро находит ближайшие к запросу (top-k). **ANN-индекс HNSW** — почти точный поиск, в разы быстрее полного перебора.

pgvector	расширение Postgres: SQL, транзакции, JOIN с данными
Qdrant	отдельный сервер: быстрые фильтры, масштаб
Chroma	встраиваемая, для прототипов

```
CREATE INDEX ON chunks
  USING hnsw (embedding vector_cosine_ops);
SELECT id, text FROM chunks
WHERE team_id = $2              -- права доступа
ORDER BY embedding <=> $1 LIMIT 5; -- ближайшие
```

`<=>` — косинусное расстояние = 1 − сходство: меньше — ближе. HNSW для `vector` — до 2000 измерений, больше — `halfvec`.

04 Чанкинг: размер и перекрытие

Документ целиком не влезет в окно, а один вектор на длинный текст **размыт**. Режем на чанки, лучше по структуре: заголовки, абзацы.



▨ перекрытие 10–20%: мысль на границе попадает в оба чанка

Слишком крупно
поиск размыт, в окно влезает мало чанков, шум, дороже

Слишком мелко
фраза без контекста, ответ из обрывков

Обычно
300–800 токенов; подбирают по оценке качества

05 Гибридный поиск и реранкинг

BM25 — по словам
коды, артикулы, имена

Векторный — по смыслу
синонимы, перефразы

↓

Объединение (RRF) — по местам в обоих списках

↓

~50 кандидатов

↓

Реранкер (cross-encoder) — читает вопрос и чанк вместе

↓

5 лучших → в LLM

Реранкер точнее, но медленнее — поэтому только по кандидатам. `ts_rank` в Postgres — не BM25.

06 RAG или fine-tuning

RAG — для знаний
документы, факты, часто меняющиеся данные; нужны источники и права доступа

Fine-tuning — для поведения
стиль, формат, узкая задача, короче промпт; новые знания так добавлять плохо

Порядок: **промптинг → RAG → дообучение**, если упёрлись в формат, стиль или цену. Можно сочетать.

RAG — не дообучение: модель не меняется, найденное только подкладывается в промпт.

07 Оценка качества RAG

Тестовый набор
50–200 реальных вопросов: эталон + нужный источник

→

Прогон RAG
найденные чанки + ответ

→

LLM-as-a-judge
оценки по чёткой рубрике + человек на выборке

Поиск (retrieval)
hit rate / recall@k — нужный чанк попал в top-k?
MRR — на каком он месте

Ответ (generation)
faithfulness — всё есть в найденных чанках?
relevance — по вопросу? **correctness** — как эталон?

Меряй поиск и ответ **отдельно**: так видно, где ломается. Любое изменение (чанкинг, модель, промпт) — прогон на **том же наборе**. В проде — фидбэк и мониторинг.

ВОПРОСЫ МЕНТОРА → Эмбеддинг 02 · Векторная БД 03 · RAG и какую проблему решает 01 · RAG-пайплайн по шагам 01 · Чанкинг: крупные и мелкие 04 · RAG или fine-tuning 06 · Оценка RAG 07

RAG: документы и эмбеддинги

01 RAG: путь факта до ответа

RAG, retrieval-augmented generation, - генерация с опорой на найденные материалы. Приложение ищет основания, добавляет их к вопросу и передаёт языковой модели. Документы остаются внешним источником: их содержимое не записывается в параметры модели при каждом запросе.

Индексация готовит источники заранее: извлечь текст → проверить структуру → разбить на чанки → добавить метаданные → построить поисковые представления. **Обработка вопроса:** определить доступ → найти кандидатов → уточнить порядок → собрать контекст → сгенерировать ответ со ссылками.

Fine-tuning, дообучение, меняет параметры модели на примерах. Помогает закрепить формат, стиль и поведение. Для замены правила «30 дней» на «45 дней» обычно обновляют источник и индекс. Дообучение само не обеспечивает удаление отдельного факта, актуальность и разграничение доступа. Эти механизмы можно сочетать.

Сквозной пример - вымышленный ассистент «Маяка». Вопрос: «Можно вернуть покупку через 40 дней?» Нужен действующий регламент v2: 45 дней, только для неиспользованного товара. Сравнения 40 ⩽ 45 недостаточно без проверки условия.

Различайте наблюдения: источник опубликован; чанк создан; найден на месте 3; вошёл в фактический запрос; правильно пересказан. Найденный документ может исчезнуть при сборке контекста. RAG не гарантирует истину: ошибаются и источники, и поиск, и генератор.

Если оснований нет, ответ должен обозначить ограничение. Вопрос «какой статус заявки сейчас?» может требовать актуального разрешённого чтения системы, а не вчерашнего текстового индекса.

02 Чанки: размер и перекрытие

Чанк - фрагмент, который ищут и передают отдельно. Маленький точнее локализует совпадение, но теряет условия; большой сохраняет связность, но смешивает темы и расходует контекст. Универсального размера нет: сравнивают на своих вопросах.

Токен - единица выбранного токенизатора; это не обязательно слово или символ. **Overlap, перекрытие**, повторяет часть текста на стыках. При размере 200 и overlap 40 шаг равен 200 − 40 = 160 токенов.

Считаем документ из 500 токенов. Первое окно: 1-200, длина 200. Второе: 161-360, длина 200. Третье: 321-500, длина 180.

Суммарно сохраняем 200 + 200 + 180 = **580 токенов**. Две зоны по 40 повторены: 580 − 500 = 80 дополнительных токенов. Три чанка одного правила не становятся тремя независимыми подтверждениями.

Делите по заголовкам, абзацам и смысловым блокам. Строке таблицы нужны названия колонок; условию - правило, которое оно ограничивает. Путь заголовков «Возвраты → Сроки → Исключения» помогает понять короткий фрагмент. Идентификатор ERR-4031 сохраняют точно: ERR-403 - другой объект.

Проверка границы: фраза «45 дней» на одной карточке, «только неиспользованный товар» на другой. Найденная первая карточка ещё не даёт полного основания. Проверьте совместную доставку, а не только наличие обоих чанков в индексе.

Overlap не исправляет потерянную колонку при извлечении PDF. Лимит модели эмбеддингов и окно генератора могут различаться; измеряйте объём нужным токенизатором.

03 Эмбеддинги и метаданные

Эмбеддинг - список чисел фиксированной длины, построенный моделью для текста. **Размерность** - число координат: (0,2; −0,4; 0,7) имеет три. Семантический поиск может сблизить «деньги назад» и «возврат оплаты» без одинаковых слов.

Близость не доказывает согласие: «возврат разрешён» и «возврат запрещён» относятся к одной теме. Отдельная координата обычно не означает конкретное человеческое понятие. Учебные двумерные примеры ниже объясняют геометрию, а не настоящие значения текстовой модели.

Совместимое пространство требует согласованной модели, версии, подготовки текста, режимов запроса/документа и нормализации. Одинаковая размерность двух моделей разрешает арифметику, но не делает их координаты общей картой. Префиксы query: и passage: нужны только там, где их предусматривает выбранная модель.

Миграция: подготовить новый индекс → перекодировать документы → сравнить поиск на фиксированных вопросах → согласованно переключить индекс и кодировщик запросов. Не смешивать старые и новые векторы. Изменился текст чанка - его старый вектор уже описывает другой текст.

Метаданные связывают число с источником: chunk_id - ID фрагмента; document_id - стабильный ID документа; version - редакция; раздел - место; статус и дата действия - применимость; область доступа - кто вправе читать.

source_updated_at означает изменение источника; indexed_at - построение записи; valid_from - начало действия правила. Свежая индексация архивной v1 не делает её актуальной. Ссылка в ответе должна вести к нужной версии и месту, а не просто к похожему документу.

RAG: документы и эмбеддинги

04 Dot product и расстояние L2

Пусть **q = (1,0)** - вопрос, **a = (2,0)** и **b = (1,1)** - два фрагмента. Координаты здесь условные. **Dot product, скалярное произведение**, умножает координаты на одинаковых местах и складывает результаты.

q·a = 1×2 + 0×0 = 2. Точка между буквами обозначает скалярное произведение.

q·b = 1×1 + 0×1 = 1. Если поиск настроен на dot product, больше - ближе; a выше b. На результат влияют направление и длина вектора.

L2 - евклидово расстояние, длина прямого отрезка между точками. Вычитаем одноимённые координаты, возводим каждую разность в квадрат, складываем и извлекаем корень. Знак √ означает квадратный корень.

L2(q,a) = √((1−2)² + (0−0)²) = √1 = 1.

L2(q,b) = √((1−1)² + (0−1)²) = √1 = 1.

Здесь ничья: для расстояния меньше - ближе. Dot и L2 дали разные результаты при тех же объектах. Некоторые системы возвращают квадрат L2 без корня: порядок сохраняется, числовой порог меняется.

Изменим только длину: вместо a возьмём (20,0). Dot станет 1×20 + 0×0 = 20. L2 станет √((1−20)² + 0²) = 19. Вектор направлен так же, но гораздо дальше от q как точка.

Рост dot с 2 до 20 не доказывает улучшение ответа. Мы изменили геометрию учебного вектора, а не проверили содержание документа. Название поля score не говорит, какая мера возвращена: сначала выясните формулу и направление сравнения.

05 Косинус и нормализация

Норма norm(v) - длина вектора v: корень из суммы квадратов координат. Для q=(1,0), a=(2,0), b=(1,1): norm(q)=√(1²+0²)=1; norm(a)=√(2²+0²)=2; norm(b)=√(1²+1²)=√2≈1,414.

Косинусное сходство cos(q,a) = (q·a) / (norm(q) × norm(a)). Делим скалярное произведение на произведение длин, чтобы сравнить направления без влияния масштаба.

Для a: cos(q,a) = 2 / (1×2) = **1**.

Для b: cos(q,b) = 1 / (1×√2) ≈ **0,707**.

У ненулевых векторов cosine лежит от −1 до 1; больше - ближе направление. Для a=(20,0) получаем 20/(1×20)=1: длина выросла, направление осталось прежним. Если система возвращает 1−cos, это расстояние, и меньше - лучше.

Нормализация делит каждую координату на длину. a превращается в (1,0), b - в (1/√2,1/√2). Теперь длины равны 1: сравниваем направления, устранив преимущество масштаба.

У единичных векторов dot равен cosine; **L2² = 2 − 2×cos**. Для нормализованных q и b: L2²≈2−1,414=0,586; L2≈0,765. Ранжирование совпадает, хотя числа и направление «лучше» различаются. Без единичных длин это обещание неверно.

Нулевой вектор имеет длину 0: cosine и нормализация не определены. Проверьте пустой ввод и выход кодировщика, явно обработайте сбой.

Cosine 0,8 не означает 80% вероятности правильного ответа. Это геометрический score. Порог полезности проверяют по разметке; после смены модели прежнее число может потерять смысл.

06 Свежесть и потерянное условие

Публикация версии: сначала подготовить и проверить новые чанки, затем согласованно включить v2 и исключить v1 из обычного текущего поиска. Удалить v1 раньше - получить окно без ответа; оставить обе активными - смешать правила 30 и 45 дней.

Максимальная дата не определяет действующий документ: более новый файл может быть черновиком. Историческому вопросу нужна редакция на указанную дату, если такая функция поддерживается и доступ разрешён. Удаление источника должно исключить его чанки и применимые кэши, а не только исходный файл.

Ответ остался «30 дней». Проверьте опубликованную v2 → активные чанки → фильтр версии и доступа → кандидатов → фактический контекст → утверждение ответа. Сегодняшний indexed_at у v1 подтверждает только время обработки. Если v2 отсутствует, увеличение числа результатов не создаст её.

Ответ разрешает использованный товар. Найдите условие «только неиспользованный» в исходнике. Если его потеряли при извлечении - исправьте извлечение. Отделили при чанкинге - пересмотрите границы. Есть в кандидате, нет в контексте - проверьте отбор и бюджет. Дошло до модели, но ответ противоречит - проверяйте генерацию.

После смены кодировщика выдача случайная. Сравните версии моделей, префиксы и нормализацию документов и вопросов. Совпавшая размерность не исключает несовместимость; ошибка может не вызвать исключения программы.

Меняйте один подтверждённый участок и повторяйте те же вопросы. Для трассы обычно достаточно ID, версий и рангов: полный внутренний текст не нужен в обычном логе. Права задаёт доверенный сервер; нельзя исправлять полноту раскрытием запрещённых материалов.

RAG: поиск и индексы

01 Лексический поиск: TF-IDF, BM25

Терм - поисковая единица после анализа текста: слово, основа или сохранённый код. Это не обязательно токен LLM. **TF** - число вхождений терма в документ; **IDF** - вес его редкости в корпусе, коллекции документов. **TF-IDF** объединяет частоту внутри документа и редкость между документами.

Мини-корпус из трёх документов: A: «возврат возврат деньги»; B: «возврат срок»; C: «err-4031 доступ». Используем учебное соглашение IDF(t)=ln(N/df(t)): t - терм, N=3 - число документов, df - сколько документов содержат терм, ln - натуральный логарифм.

«Возврат» есть в A и B: df=2, IDF=ln(3/2)≈0,405. «Деньги» и «err-4031» встречаются в одном документе: df=1, IDF=ln(3)≈1,099.

Вес «возврат» в A: TF×IDF=2×0,405=0,810; в B: 1×0,405=0,405. Код в C: 1×1,099=1,099. Терм во всех документах получил бы ln(1)=0. Реальные реализации могут сглаживать и нормализовать иначе.

BM25 ограничивает выигрыш от повторов и учитывает длину. При k1=1,2, где k1 управляет насыщением, частотный множитель без поправки длины: TF×(k1+1)/(TF+k1).

TF=1: 2,2/2,2=1. TF=2: 4,4/3,2=1,375. TF=10: 22/11,2≈1,964. Десять повторов не дают десятикратной пользы. Это множитель, не полный score.

Поправка заменяет k1 в знаменателе на k1×(1−b+b×dl/avgdl): b - сила поправки; dl - длина документа в термах; avgdl - средняя длина. При b=0 поправки нет. BM25 не создаёт совпадение синонимов без общего терма или дополнительной обработки.

02 Sparse, dense и фильтры

Sparse, разреженное представление: координат много, большинство у конкретного текста нулевые. В словарном варианте координата соответствует терму. **Dense**, плотное представление: фиксированный вектор, обычно с преимущественно ненулевыми координатами. Обучаемый sparse тоже возможен; sparse не равен исключительно BM25.

Инвертированный индекс хранит для терма список документов. По «возврат» сразу смотрим A и B. Dense-поиск кодирует вопрос и ищет близкие векторы в совместимом пространстве. Для точного ERR-4031 полезен лексический сигнал; для «деньги назад» - семантический. Ни один сам не гарантирует правильное правило.

Векторное хранилище связывает эмбеддинги с ID и метаданными. **Фильтр** ограничивает кандидатов по организации, доступу, языку, статусу, версии и дате действия. Для текущего вопроса архив «30 дней» не должен конкурировать с действующим «45 дней». Исторический вопрос требует другого условия применимости.

Права проверяют до раскрытия: запрещённый текст не должен попасть пользователю, внешнему реранкеру, LLM, общему кэшу и обычным логам. Область доступа выводит доверенный сервер, а не произвольный team_id браузера. Удалить ссылку после передачи текста модели уже недостаточно.

Если фильтровать после ограниченного top-5 и четыре результата запрещены, останется один, хотя дальше могли быть полезные разрешённые. Нужны согласованные поиск, фильтрация и пополнение кандидатов. Для сравнения exact и ANN разрешённый корпус должен совпадать.

Ключ кэша учитывает доступ и версию. Метаданные тоже могут раскрывать внутреннюю информацию. Одинаковый сервер может поддерживать лексический и векторный поиск: две стратегии не обязательно требуют двух отдельных систем.

03 Exact, ANN, HNSW и IVF

Exact сравнивает запрос со всеми допустимыми векторами и выбирает top-k, лучшие k объектов по метрике. **ANN**, approximate nearest neighbors, сокращает работу, допуская пропуск точных соседей. Точный геометрический сосед ещё не обязательно полезный документ.

ANN recall@k - доля exact top-k, попавшая в ANN top-k. Exact: [A,B,C,D,E]; ANN: [A,B,C,F,G]. При k=5 общих три: ANN recall@5=3/5=0,6. Это точность приближения индекса; Retrieval Recall сравнивает с экспертной полезностью. Exact может иметь ANN recall=1 и бесполезную выдачу.

HNSW - многоуровневый граф: вершины - векторы, связи ведут к соседям. Верхние уровни помогают найти перспективную область, нижние уточняют поиск. Три настройки отвечают за разные работы.

M - связанность графа: больше связей обычно требует больше памяти и работы построения. **efConstruction** - ширина исследования при построении; более тщательная сборка дороже. **efSearch** - ширина исследования при запросе; рост обычно улучшает полноту ценой задержки. Top-k при этом можно оставить равным 5. Изменение M и efConstruction обычно требует перестроения; детали зависят от движка.

IVF группирует пространство вокруг центров. **nlist** - число групп, **nprobe** - сколько групп просматривают на запрос. Из 10 000 векторов в 100 примерно равных группах nprobe=1 даёт около 100 проверок, nprobe=10 - около 1000, плюс выбор центров.

Больше nlist при прежнем nprobe не гарантирует полноту: просматривается меньшая доля корпуса. Все группы с точными несжатыми векторами дают полный перебор; квантование добавляет отдельную ошибку. Сравнивайте качество и задержку на одном наборе; рост ширины не исправляет несовместимые эмбеддинги.

RAG: поиск и индексы

04 Гибридный поиск и RRF

Гибридный поиск объединяет, например, BM25 и dense. Складывать BM25=12 и cosine=0,8 без обоснования нельзя: разные шкалы. **RRF, reciprocal rank fusion**, использует позиции, а не сырые scores. Позиции начинаются с 1; у фрагмента должен быть стабильный ID.

RRF(d) = сумма 1/(c + rank) по спискам, содержащим d. Здесь d - фрагмент; rank - позиция в конкретном списке; c - константа сглаживания. Возьмём c=60. Отсутствующий фрагмент даёт вклад **0**, а не rank=0.

Позиция	BM25	Dense
1	A	B
2	B	C
3	D	A

A: есть на местах 1 и 3. RRF(A)=1/61+1/63≈0,016393+0,015873=0,032266.

B: места 2 и 1. RRF(B)=1/62+1/61≈0,016129+0,016393=0,032522.

C: отсутствует в BM25, второе место в dense. RRF(C)=0+1/62≈0,016129.

D: третье место в BM25, отсутствует в dense. RRF(D)=1/63+0≈0,015873.

Общий порядок: B, A, C, D. В чуть выше A благодаря сочетанию позиций. Это порядок объединения, а не доказательство истинности или вероятности правильного ответа. Если вместо нулевого вклада добавить 1/60, отсутствие ошибочно станет преимуществом.

Дедупликация: A из двух списков становится одной записью с двумя вкладами. Перекрывающиеся чанки с разными ID требуют отдельной обработки близких дублей. Нельзя выбрасывать единственный чанк с исключением только из-за похожести.

с сглаживает преимущество первых мест; 60 - соглашение примера. Глубина входных списков влияет на результат: фрагмент за границей списка не получает вклад. Качество гибрида подтверждают отдельными группами вопросов, включая коды и перефразировки.

05 Реранкер и бюджет контекста

Bi-encoder кодирует вопрос и документ отдельно. Векторы документов готовят заранее, на запросе считают один вектор вопроса. Это быстрый широкий отбор; тонкое условие может потеряться в сжатом представлении.

Cross-encoder обрабатывает пару «вопрос + фрагмент» совместно и оценивает соответствие. Это реранкер: замечает взаимодействие точного кода, отрицания и условий, но требует обработки каждой пары. Для 50 кандидатов - 50 пар; фактическая стоимость зависит от модели, длины и пакетной обработки.

Потолок кандидатов: реранкер переставляет только полученное. Если важное условие стоит на месте 25, а вход ограничен первыми 20, более точная модель его не восстановит. Сначала проверяют полноту отбора, затем первые позиции.

Разделяйте три числа: получить 40 кандидатов → переранжировать 30 после фильтров и дедупликации → отправить 5 в контекст. Поиск шире не обязан увеличивать запрос генератору, но влияет на задержку и стоимость.

Считаем бюджет. Окно модели - 8000 токенов. Вопрос и инструкции - 1000. Резерв ответа - 1500. Под документы остаётся **8000−1000−1500=5500**.

Тело чанка - 900 токенов; заголовок, ID и ссылка - ещё 50. Один готовый блок занимает **950**.

Шесть блоков: 6×950=5700, превышение на 200. Пять: 5×950=4750, остаётся 750. Значит в этих условиях помещаются **пять**, а не шесть. У настоящих чанков длины различаются: считают окончательно собранный запрос.

Бюджет расходуют повторы и противоречивые редакции. Сохраняйте правило с исключением, а не просто самые похожие строки. Если основания не помещаются, выбирайте стратегию отбора или уточняйте вопрос; молчаливое усечение условия способно изменить ответ.

06 Диагностика поисковой цепочки

Начните с покрытия: нужный документ опубликован, проиндексирован, действует и разрешён пользователю? Если инструкции ERR-4031 в доступном корпусе нет, настройка графа её не создаст. Правильное ограничение доступа нельзя считать поводом раскрыть закрытый документ.

Лексический путь не находит код. Проверьте анализатор: сохранил ли ERR-4031 целиком, не превратил ли в общий 403? Сравните точный запрос с перефразировкой. Отсутствие общего терма не объясняется параметрами HNSW.

Dense exact плох по смыслу. Проверьте совместимость модели, версии, префиксов, нормализации; затем чанки и соответствие типа поиска задаче. Exact точно воспроизводит геометрию, даже если она неудачна для нужного кода.

Exact хорош, ANN теряет соседа. При одинаковых векторах и фильтрах измерьте ANN recall и задержку. Для HNSW меняйте efSearch; для IVF - nprobe. Это разные индексы: нельзя автоматически «поднять оба параметра» у любой системы.

Кандидат найден, но исчез позже. Сохраните его ID и позиции: исходная выдача → RRF → вход реранкера → финальный порядок → контекст. Проверьте отсечение списков, объединение ID, дедупликацию, порядок и токены. Если ERR-4031 первый после реранкера, но отсутствует в промпте, подтверждённый разрыв находится в сборке контекста.

Контекст полный, ответ неверен. Сравните утверждения с источником: дата, число, условие, отрицание и ссылка. Тогда проверяют генерацию, а не автоматически перестраивают индекс.

Для эксперимента фиксируйте корпус, права, вопросы и бюджет. Меняйте один участок, заранее запишите ожидаемый результат и измерьте качество вместе со стоимостью. Средний выигрыш может скрыть полный провал на редких критичных кодах.

RAG: оценка и диагностика

01 Набор оценки и релевантность

Evaluation set, набор оценки, - фиксированные вопросы с разрешённым корпусом, ожидаемыми основаниями и критериями ответа. Качество определяют до эксперимента: «лучше» может означать найти больше оснований, поднять первое полезное, верно передать условие или сократить задержку.

Релевантность - полезность материала для конкретного вопроса по инструкции разметки. Это не score модели. Для «можно вернуть через 40 дней сегодня?» нужен действующий v2: 45 дней, только неиспользованный товар. Для исторического вопроса может понадобиться v1: 30 дней. Дата и права меняют допустимый эталон.

Единица оценки: документ, чанк или факт. Десять перекрывающихся чанков одного правила не становятся десятью независимыми источниками. При документной оценке объединяют по document_id; при чанковой фиксируют разбиение и правила зачёта. Нельзя сравнивать разные знаменатели как одну задачу.

Сквозной пример: во всём разрешённом корпусе четыре релевантных фрагмента. Поиск вернул три различных фрагмента с метками **[0,1,1]** по порядку. 0 - бесполезный, 1 - полезный; дубликатов нет. Два полезных найдены, ещё два остались в корпусе.

Для полноты нужны метки не только найденного, но и сведения обо всех релевантных. Неполная разметка ограничивает выводы: «не размечен» не обязательно означает «бесполезен». Проверяйте новые ID вручную, объединяйте кандидатов разных систем и сообщайте охват.

Зафиксируйте снимок документов, дату действия, доступ, вопросы, эталонные утверждения, глубину выдачи и обработку пограничных случаев. Меняется корпус - меняется задача. Отдельные группы: точные коды ERR-4031, перефразировки, условия, история и вопросы без ответа.

02 Precision, Recall, RR и MRR

k - число верхних оцениваемых позиций; **r_k** - число полезных среди них; **R** - число всех релевантных в разрешённом корпусе. **Precision@k = r_k/k**, чистота выдачи. **Recall@k = r_k/R**, доля найденного полезного.

Для **[0,1,1]**: k=3; r_k=2; R=4.

Precision@3 = 2/3 ≈ 0,667. Две трети выбранного полезны. Знаменатель - число позиций, а не размер корпуса.

Recall@3 = 2/4 = 0,5. Нашли половину всех полезных. Высокая чистота не гарантирует полноту. На глубине 1 обе метрики равны 0: единственный переданный генератору чанк оказался бы бесполезным.

RR, reciprocal rank, - обратная позиция первого полезного: RR=1/rank_first. rank_first - позиция начиная с 1. В примере первое полезное на месте 2: **RR=1/2=0,5**. Если полезного в оцениваемом списке нет, RR=0.

MRR, mean reciprocal rank, - среднее RR по вопросам. Для трёх вопросов с RR 0,5; 1; 0: **MRR=(0,5+1+0)/3=0,5**. RR относится к одному вопросу, MRR - к набору. У [1,0,0] и [1,1,1] одинаковый RR=1: поздние полезные он не учитывает.

Больше - лучше у всех этих метрик, но каждая отвечает на свой вопрос. Усечённую глубину RR/MRR указывают явно.

Короткая выдача: договоримся считать отсутствующие позиции бесполезными. Вернулись три при top-5, два полезны: Precision@5=2/5, а не 2/3. Пустая выдача при R>0 даёт Recall=0. **ANN recall** имеет другой эталон - exact-соседей, а не экспертную полезность.

03 NDCG: счёт по шагам

NDCG сравнивает полезность порядка с идеалом, уменьшая вес дальних мест. **rel** - метка релевантности; **gain(rel)=2^rel−1** - числовая полезность; знак ^ обозначает степень. Для rel 0, 1, 2 получаем gain 0, 1, 3. Это выбранная шкала, не природный закон.

DCG@k = сумма gain(rel_i)/log2(i+1) по позициям i от 1 до k. rel_i - метка на позиции i; log2 - логарифм по основанию 2. **IDCG** - DCG идеального порядка. **NDCG@k = DCG@k/IDCG@k**.

Шаг 1. Скидки для k=3. Место 1: log2(2)=1. Место 2: log2(3)≈1,585. Место 3: log2(4)=2.

Шаг 2. Выдача [0,1,1]. Полезности: [0,1,1]. DCG@3=0/1+1/1,585+1/2≈0+0,63093+0,5=**1,13093**.

Шаг 3. Идеал по корпусу. Всего четыре релевантных, поэтому идеальный top-3 - [1,1,1]. IDCG@3=1/1+1/1,585+1/2≈**2,13093**. Не переставляйте только найденные [0,1,1]: так потерянные основания исчезнут из эталона.

Шаг 4. Деление. NDCG@3≈1,13093/2,13093≈**0,53072**. Это доля идеальной взвешенной полезности, а не вероятность верного ответа. Больше - лучше; 1 соответствует идеальному результату по выбранному протоколу.

Зачем градации? При rel=2 полностью полезное имеет gain=3. Для [2,1,0] DCG≈3+0,63093=3,63093; для [1,2,0] - 1+3/1,585≈2,89279. Сильное основание на первом месте выгоднее.

При сравнении реализаций согласуйте gain, скидку, эталон, k и ничьи. Некоторые инструменты принимают готовые полезности и не вычисляют 2^rel−1 сами. При IDCG=0 обычное деление не определено: требуется отдельное соглашение.

RAG: оценка и диагностика

04 Контекст, истина и ответ

Faithfulness, верность контексту: существенные утверждения подтверждены переданными фрагментами и не противоречат им. **Фактическая правильность:** утверждения соответствуют действующему проверенному правилу. **Answer relevance:** текст решает заданный вопрос. **Context relevance:** выбранные фрагменты полезны для его решения.

«30 дней» по архиву v1 может быть faithful, но неверно для сегодняшнего правила 45 дней. **«45 дней» без срока в контексте** может случайно совпасть с истиной, но не иметь опоры. Правильный рассказ о возвратах на вопрос про ERR-4031 фактически точен и нерелевантен вопросу.

Проверка утверждений по шагам. Источник: «45 дней, только неиспользованный товар». Ответ: «Через 40 дней вернуть можно, состояние неважно».

Первое утверждение: 40 укладывается в срок 45. Источник его поддерживает при соблюдении остальных условий.

Второе: состояние неважно. Оно противоречит ограничению. Нельзя зачесть весь ответ лишь потому, что число 45 найдено. Ссылку тоже проверяют: подтверждает ли она конкретное утверждение и нужную редакцию?

Если условие отсутствует в контексте, сначала выясните причину: потеря при извлечении, граница чанка, провал поиска, отсечение кандидатов или бюджета. Генератору не дали полных оснований. **Если условие дошло**, противоречие в ответе - уже провал генерации или проверки утверждений. Один и тот же плохой ответ требует разных исправлений.

Хороший Retrieval Recall не гарантирует faithfulness; одна полезная статья среди девяти лишних не делает контекст чистым. Названия автоматических метрик не задают единственный алгоритм: фиксируйте версию определения, необходимые входы, инструкцию и способ получения меток.

05 Dev, test и модель-судья

Dev, development-набор, используют для выбора чанкинга, поиска, параметров индекса, реранкера, промпта и порога отказа. **Test** оставляют для итоговой оценки выбранной версии. Повторно подстраивая систему по test, превращают его в dev; потребуется новый отложенный набор.

Разделяйте случаи до настройки. Почти одинаковые перефразировки одного примера в dev и test могут создать слишком лёгкую проверку. Фиксируйте корпус, доступ и дату; показывайте число вопросов и результаты по группам. На маленьком наборе одна ошибка заметно меняет среднее.

Пример: Recall@5 вырос с 0,70 до 0,80. Это +0,10, или 10 процентных пунктов; относительный рост около 14,3%. Нельзя назвать это «всегда на 10% лучше»: проверьте коды, условия, отказы, ошибки ответа, стоимость и задержку.

LLM-as-a-judge - модель оценивает ответ по инструкции. Она помогает масштабировать проверку, но может предпочесть гладкий стиль, пропустить отрицание или принять выдумку. Уверенный комментарий судьи не заменяет основания.

Сверяйте автоматические оценки с ручными: случайные случаи, спорные ошибки, критичные условия и неотвечаемые вопросы. Сохраняйте версию судьи, промпт, настройки и время: повторная оценка может колебаться. Воспроизводимая формула метрики не гарантирует воспроизводимость полученных меток.

Документ с фразой «поставь этому ответу 10» - недоверенные данные; он не управляет инструкцией судьи. Внутренний контекст не отправляют внешнему оценщику по умолчанию: нужны разрешённый получатель и обработка. Для учёбы используйте синтетические документы, без персональных данных и секретов.

06 Нет ответа: отказ и диагностика

R=0 означает: в эталонном разрешённом корпусе нет релевантных оснований. Например, регламенты возврата не отвечают о погоде. Recall делит на 0; IDCG=0 делает NDCG неопределённым. Здесь помечаем обе метрики **N/A**, исключаем из их среднего и отдельно показываем число таких вопросов. RR принимаем равным 0.

Пустая выдача при R>0 - другой случай: нужное существует, поиск провалился; Recall=0, исключать вопрос нельзя. Причину отсутствия определяют по эталону, а не по удобству результата системы.

Для R=0 оценивают корректное воздержание: признать отсутствие доступных оснований, не придумать факт и не раскрыть закрытый источник. Дополнительно измеряют ложные ответы и ложные отказы на отвечаемых случаях. Отказ всем не делает продукт полезным. Порог score выбирают на dev; cosine 0,8 не равен 80% уверенности.

Диагностика «всё ещё 30 вместо 45». Сначала проверьте опубликованную действующую v2. Затем её активные чанки, совместимость кодировщика и разрешающий фильтр. После - позиции в лексическом, exact, ANN и объединённом списках. Далее - фактический контекст с условием. Наконец - каждое утверждение ответа.

Первый подтверждённый разрыв задаёт изменение: нет v2 - публикация; exact находит, ANN нет - ширина поиска; реранкер находит, контекст нет - сборка; полный контекст есть, условие нарушено - генерация.

Меняйте один этап и повторяйте фиксированную оценку с задержкой и стоимостью. Согласованная миграция кодировщика и индекса считается одной версией эксперимента. Для трассы достаточно минимальных ID, версий и рангов с ограниченным доступом; полный внутренний текст в обычном логе не нужен.